

Обработка и анализ данных

EDA

Импортируем все необходимые библиотеки, зафиксируем seed и загрузим данные

```
In [460... import random
import numpy as np
import pandas as pd
from rapidfuzz import fuzz, process, utils
import matplotlib.pyplot as plt
import seaborn as sns
from sklearn.feature_extraction.text import CountVectorizer, TfidfVectorizer
from sklearn.decomposition import TruncatedSVD
from sklearn.cluster import KMeans
import re
import pymorphy3

SEED = 42
random.seed(SEED)
np.random.seed(SEED)
```

```
In [461... df = pd.read_csv('data.csv')
print(df.head())
print(df.tail())
```

	query_text
0	гетры y2k черные широкие
1	шторы в спальню 100*210
2	big bcaa
3	щетки на болгарку диолд
4	цинк и медь хелат
	query_text
19995	d1 barracuda
19996	нож финка нквд складной
19997	кепка для мальчика 44
19998	мультифункциональная мойка
19999	умная лампочка яндекс gu10

Перед проведением анализа данных, отметим некоторые специфики e-commerce данных:

1. Опечатки
2. Сленг, сокращения и англицизмы (транслитерация)
3. Определенный формат единиц измерения (размер товаров, вес и т.д.)
4. В потоке слов необходимо верно различать их атрибуты и значимость
5. Дубликаты запросов

Но для начала проанализируем данные, посмотрим на структуру

Разбор структуры по статистикам: длина запросов, количество слов, наличие латинских букв и цифр

Приведем данные к удобному и чистому виду

In [462...

```
df = df.dropna(subset=['query_text'])
df['query_clean'] = df['query_text'].astype(str).str.lower().str.strip()
```

Оцениваем дубликаты и пропуски

In [463...

```
total_queries = len(df)
unique_queries = df['query_clean'].nunique()
print(f"Уникальных запросов: {unique_queries} ({unique_queries / total_queries:.2%})")
print(f"Пропущенных значений в 'query_clean': {df['query_clean'].isna().sum()} ({df['query_clean'].isna().sum() / total_queries:.2%})")
```

Уникальных запросов: 20000 (100.0% от общего числа)

Пропущенных значений в 'query_clean': 0 (0.0% от общего числа)

Видим что очевидных дубликатов и пропусков нет. Однако с учетом специфики данных в дальнейшем стоит уделить внимание обработке опечаток, сленга и транслитерации, а также анализу формата единиц измерения в запросах.

Рассмотрим базовые статистики

In [464...

```
# Вычисление базовых признаков, добавляем приведение к типу строка для обработки
df['char_len'] = df['query_clean'].astype(str).apply(len) # Длина строки в символах
df['word_count'] = df['query_clean'].astype(str).apply(lambda x: len(x.split()))
df['has_digits'] = df['query_clean'].astype(str).apply(lambda x: int(any(c.isdigit() for c in x)))
df['has_latin'] = df['query_clean'].astype(str).apply(lambda x: int(bool(re.search(r'[a-zA-Z]', x))))

print('\n          Первые 5 строк с новыми признаками:')
print(df.head())
print('\n          Описательная статистика:')
print(df.describe())
```

Первые 5 строк с новыми признаками:

	query_text	query_clean	char_len	word_count	\
0	гетры у2k черные широкие	гетры у2k черные широкие	24	4	
1	шторы в спальню 100*210	шторы в спальню 100*210	23	4	
2	big bcaa	big bcaa	8	2	
3	щетки на болгарку диолд	щетки на болгарку диолд	23	4	
4	цинк и медь хелат	цинк и медь хелат	17	4	

	has_digits	has_latin
0	1	1
1	1	0
2	0	1
3	0	0
4	0	0

Описательная статистика:

	char_len	word_count	has_digits	has_latin
count	20000.000000	20000.000000	20000.000000	20000.000000
mean	25.565650	3.899350	0.254600	0.214200
std	10.187113	1.545878	0.435647	0.410276
min	2.000000	1.000000	0.000000	0.000000
25%	18.000000	3.000000	0.000000	0.000000
50%	25.000000	4.000000	0.000000	0.000000
75%	32.000000	5.000000	1.000000	0.000000
max	101.000000	20.000000	1.000000	1.000000

Визуализируем полученные статистики

In [465...

```
sns.set_style('whitegrid')

fig, ax = plt.subplots(2, 2, figsize=(10, 8))
ax = ax.flatten()

sns.histplot(df['char_len'], bins=50, ax=ax[0], kde=True)
ax[0].set_title('Распределение длины строк в данных')
ax[0].set_xlabel('Длина строки')
ax[0].set_ylabel('Количество строк')
ax[0].grid(True, alpha=0.5)

sns.barplot(x='word_count', y='count', data=df.groupby('word_count').size().reset_index())
ax[1].set_title('Распределение количества слов в данных')
ax[1].set_xlabel('Количество слов')
ax[1].set_ylabel('Количество строк')
ax[1].grid(True, alpha=0.5)

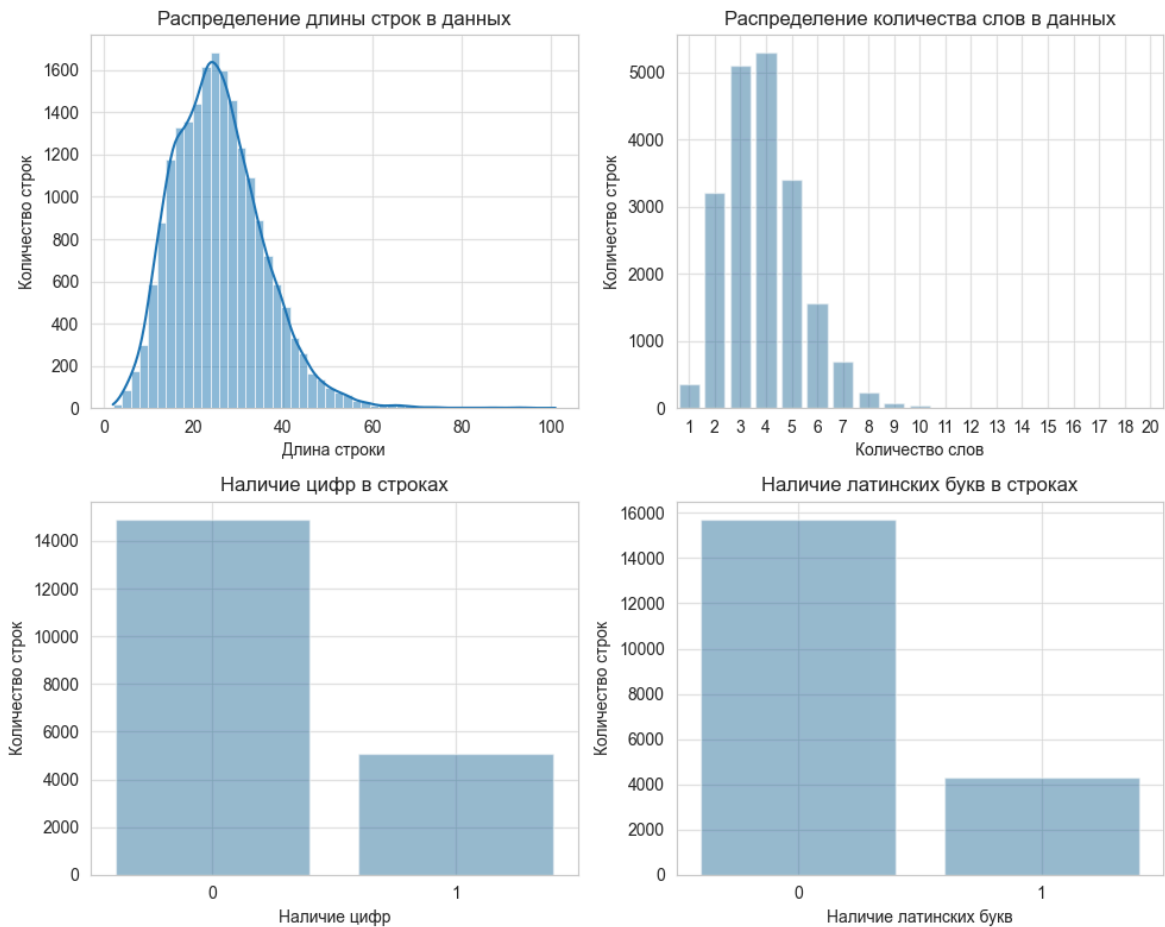
sns.barplot(x='has_digits', y='count', data=df.groupby('has_digits').size().reset_index())
ax[2].set_title('Наличие цифр в строках')
ax[2].set_xlabel('Наличие цифр')
ax[2].set_ylabel('Количество строк')
ax[2].grid(True, alpha=0.5)

sns.barplot(x='has_latin', y='count', data=df.groupby('has_latin').size().reset_index())
ax[3].set_title('Наличие латинских букв в строках')
ax[3].set_xlabel('Наличие латинских букв')
ax[3].set_ylabel('Количество строк')
ax[3].grid(True, alpha=0.5)

plt.grid(True)
plt.tight_layout()
```

```
plt.show()
```

```
print(df.describe())
```



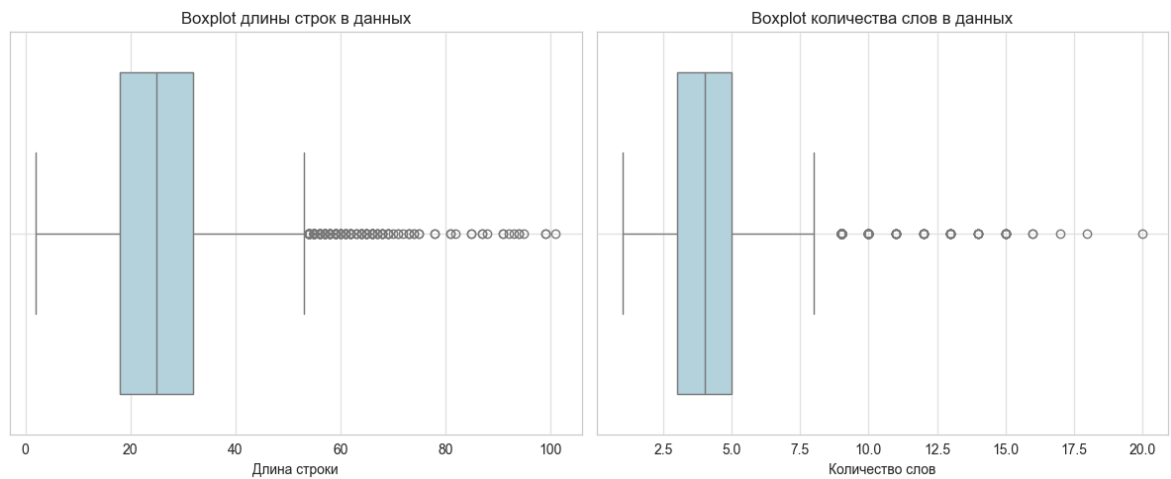
	char_len	word_count	has_digits	has_latin
count	20000.000000	20000.000000	20000.000000	20000.000000
mean	25.565650	3.899350	0.254600	0.214200
std	10.187113	1.545878	0.435647	0.410276
min	2.000000	1.000000	0.000000	0.000000
25%	18.000000	3.000000	0.000000	0.000000
50%	25.000000	4.000000	0.000000	0.000000
75%	32.000000	5.000000	1.000000	0.000000
max	101.000000	20.000000	1.000000	1.000000

In [466...

```
fig, ax = plt.subplots(1, 2, figsize=(12, 5))
ax = ax.flatten()

sns.boxplot(x=df['char_len'], color='lightblue', ax=ax[0])
ax[0].set_title('Boxplot длины строк в данных')
ax[0].set_xlabel('Длина строки')
ax[0].grid(True, alpha=0.5)
plt.grid(True)

sns.boxplot(x=df['word_count'], color='lightblue', ax=ax[1])
ax[1].set_title('Boxplot количества слов в данных')
ax[1].set_xlabel('Количество слов')
ax[1].grid(True, alpha=0.5)
plt.grid(True)
plt.tight_layout()
```



Посмотрим на пример наиболее длинного запроса

In [467...

```
argmax_len = df['char_len'].idxmax()
print(f"Максимальная длина строки: {df['char_len'].max()}")
pd.set_option('display.max_colwidth', None)
print(f"Строка с максимальной длиной: {df.loc[argmax_len, 'query_clean']}")
```

Максимальная длина строки: 101

Строка с максимальной длиной: citroën немо автофургон / микроавтобус (aa_) 1.4 ав
тофургон / микроавтобус бензин 73 2008- kfv (tu3a)

Видим структуру данных:

1. Медианное значение количества символов в запросе - 25, что может указывать на краткие запросы, возможно, состоящие из 2-4 слов. При этом количество символов в запросах преимущественно варьируется от 10 до 70. Однако намного более интерпретируемый показатель - это количество слов в запросе, так как он более информативен для понимания структуры запросов и их обработки.
2. Медианное значение количества слов в запросе - 4, что подтверждает предположение о краткости запросов. В целом большинство запросов имеют от 2 до 6 слов
3. По примеру наиболее длинного запроса можем сделать предположение, что некоторые запросы могут представлять собой длинные названия, текст, скопированный из описания товара.
4. Хотя количество латинских букв составляет меньшую часть запросов (примерно четверть), однако они присутствуют в данных, что может указывать на наличие англицизмов, названия брендов.
5. Сделаем предположение, что наличие цифр в запросах может указывать на запросы, связанные с размерами товаров, количеством, моделью и т.д. Потому для четверти запросов важно сохранить цифры при обработке текста, чтобы не потерять важную информацию.

N-граммы

Рассмотрим еще раз формат имеющейся таблицы данных

In [468... `df.head()`

Out[468...

	query_text	query_clean	char_len	word_count	has_digits	has_latin
0	гетры у2k черные широкие	гетры у2k черные широкие	24	4	1	1
1	шторы в спальню 100*210	шторы в спальню 100*210	23	4	1	0
2	big bcaa	big bcaa	8	2	0	1
3	щетки на болгарку диолд	щетки на болгарку диолд	23	4	0	0
4	цинк и медь хелат	цинк и медь хелат	17	4	0	0

Проанализируем запросы с помощью N-грамм, чтобы выявить наиболее частые сочетания слов и понять структуру запросов. Это поможет в дальнейшем при построении модели для обработки запросов и извлечения из них информации.

In [469...

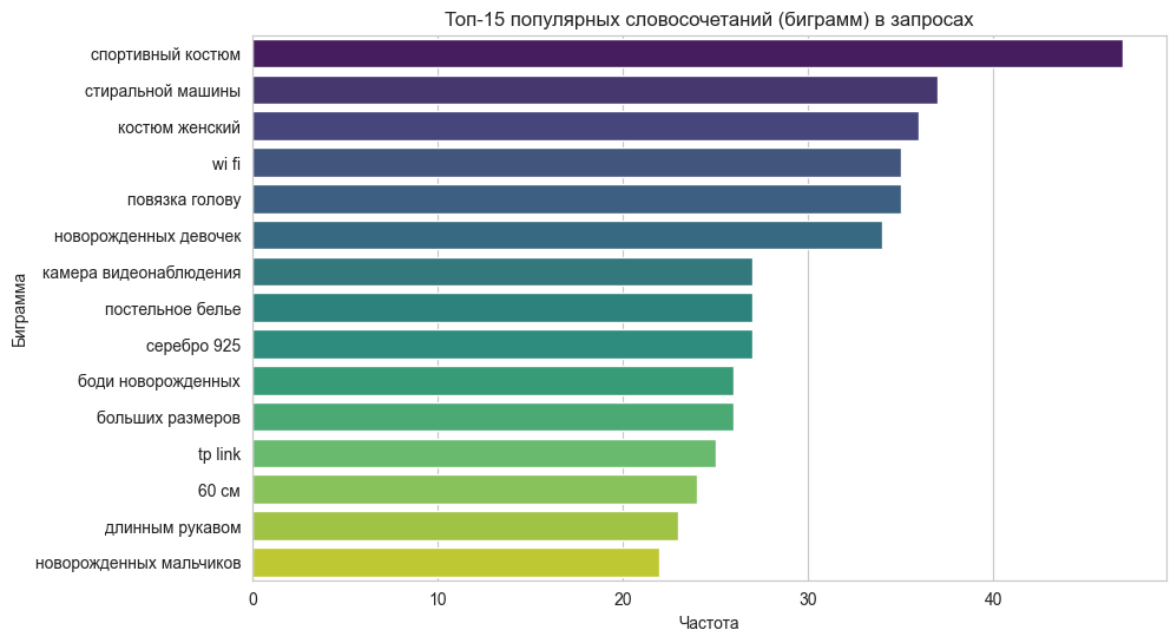
```
# список ненужных/неинформативных слов
unnec = ['в', 'на', 'для', 'с', 'из', 'под', 'и', 'к', 'за',
         'по', 'о', 'у', 'от', 'до', 'при', 'без', 'над', 'об', 'про', 'кроме',

def get_top_ngrams(data, n=None, g=1):
    vec = CountVectorizer(ngram_range=(g, g), stop_words=unnec)
    bag_of_words = vec.fit_transform(data)
    sum_words = bag_of_words.sum(axis=0).A1
    words_freq = sorted(zip(vec.get_feature_names_out(), sum_words), key=lambda
    return words_freq[:n])
```

In [470...

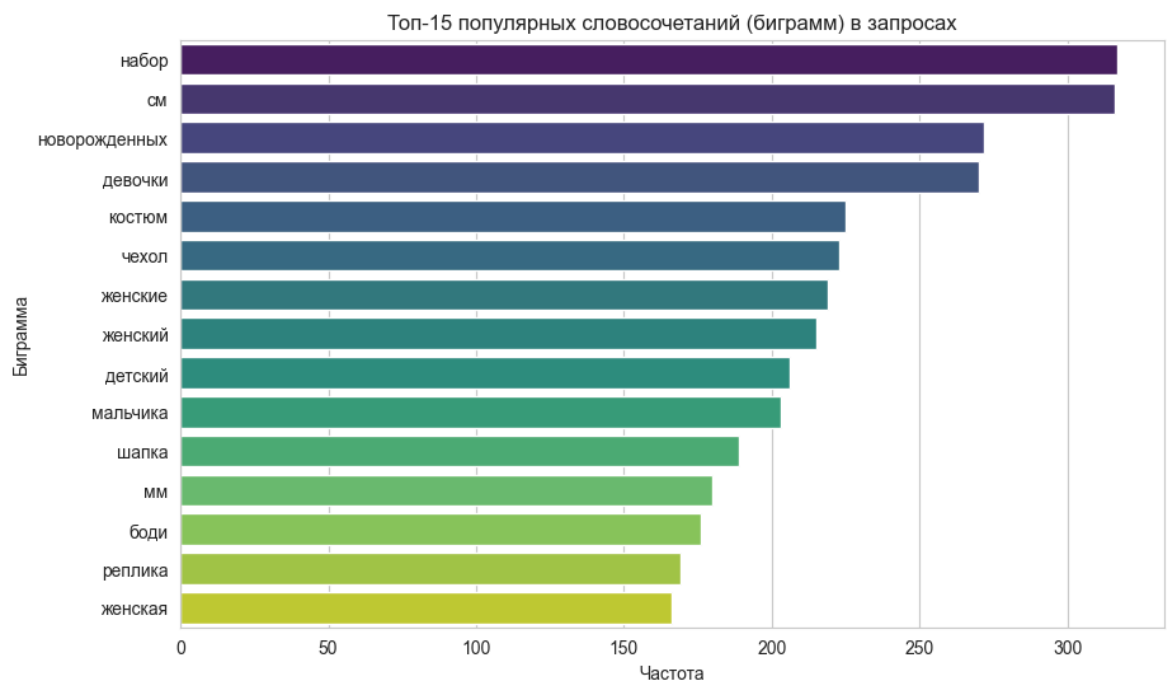
```
top_bigrams = get_top_ngrams(df['query_clean'], n=15, g=2)
df_bigrams = pd.DataFrame(top_bigrams, columns=['Биграмма', 'Частота'])

plt.figure(figsize=(10, 6))
sns.barplot(x='Частота', y='Биграмма', data=df_bigrams, palette='viridis', legend=
plt.title('Топ-15 популярных словосочетаний (биграмм) в запросах')
plt.show()
```



```
In [471... top_bigrams = get_top_ngrams(df['query_clean'], n=15, g=1)
df_bigrams = pd.DataFrame(top_bigrams, columns=['Биграмма', 'Частота'])

plt.figure(figsize=(10, 6))
sns.barplot(x='Частота', y='Биграмма', data=df_bigrams, palette='viridis', legend=
plt.title('Топ-15 популярных словосочетаний (биграмм) в запросах')
plt.show()
```



Кластеризация

Попробуем кластеризовать запросы простым способом, однако с учетом проклятия размерности и специфики данных

```
In [472... # Векторизуем слова по принципу TF-IDF, ограничив количество признаков и исключи
tfidf = TfidfVectorizer(max_features=1000, stop_words=unec)
vec_tfidf = tfidf.fit_transform(df['query_clean'])
# С помощью простого сингулярного разложения понижаем размерность
```

```

trsvd = TruncatedSVD(n_components=50, random_state=SEED)
vec_trsvd = trsvd.fit_transform(vec_tfidf)
# Кластеризация с KMeans
n_clusters = 10
kmeans = KMeans(n_clusters=n_clusters, random_state=SEED, n_init = 10)
labels = kmeans.fit_predict(vec_trsvd)

```

In [473...

```

df['cluster'] = labels
print("Ключевые слова по кластерам:")
centroids = trsvd.inverse_transform(kmeans.cluster_centers_)
order_centroids = centroids.argsort()[::-1]
names = tfidf.get_feature_names_out()

for i in range(n_clusters):
    cluster_words = [names[j] for j in order_centroids[i, :7]]
    print(f"\n Кластер {i}: ", ", ".join(cluster_words))

```

Ключевые слова по кластерам:

Кластер 0: см, девочки, мм, детский, тюль, большой, мальчика

Кластер 1: волос, ободок, резинки, заколка, набор, резинка, краска

Кластер 2: костюм, женский, спортивный, медицинский, мужской, девочки, черный

Кластер 3: новорожденных, девочек, шапка, комбинезон, мальчиков, носки, повязка

Кластер 4: кресло, мешок, подвесное, шезлонг, качалка, кровать, мягкое

Кластер 5: женские, кроссовки, очки, трусы, черные, зимние, колготки

Кластер 6: набор, девочки, подарочный, мебели, шт, дома, посуды

Кластер 7: чехол, pro, iphone, samsung, гитары, 15, airpods

Кластер 8: боди, новорожденных, рукавом, малышей, мальчиков, малыша, девочки

Кластер 9: реплика, кошек, xiaomi, kugoo, принтер, машина, pro

Отсюда видим, что запросы можно условно разделить на 10 кластеров, каждый из которых характеризуется определенными ключевыми словами. В частности рассмотрим некоторые кластеры:

1. Кластер 9 выделил запросы, связанные с техникой: гаджеты (xiaomi, реплика, pro - вероятно, речь идет о модели телефона apple, либо в целом некоторая версия какого-либо устройства), однако слова кошек и машина - довольно многозначные, так как могут относиться к разным категориям товаров (кошки - может быть игрушка и животное, также может говорить о принадлежности к определенной категории (для кого/чего создан товар), а машина - может быть игрушка или бытовая техника), потому нельзя точно сказать какие именно товары выделил этот кластер. Хотя с учетом специфики e-commerce данных, можно предположить, что машина относится к бытовой технике, а кошки - к товарам для животных: например элетронный прибор для кошек

2. Кластер 1 выделил товары для волос, при этом все слова хорошо описывают данную категорию
3. Кластер 7 выделил смартфоны и аксессуары, при этом слова хорошо описывают данную категорию, однако слово гитара выбивается из общего контекста: вероятно это связано с словосочетанием чехол для гитары
4. ...

В целом видим, что не все кластеры хорошо интерпретируются, во многих случаях разбиения по ключевым словам попадают выпадающие из общей логики слова, некоторые кластеры приписали себе такие слова как мм и см, что также наталкивает нас на мысль о том, что необходимо уделить внимание обработке единиц измерения в запросах, так как они могут быть важными для понимания структуры запросов и их кластеризации. В целом, в качестве первой попытки кластеризации, мы получили достаточно логичные кластеры, а также получили понимание исследуемых данных и их специфики

Извлечение атрибутов

Целевые атрибуты

Поставим задачу извлечения из запросов следующих атрибутов:

1. Цвет
2. Размер
3. Бренд
4. Целевая группа
5. Категория товара

Воспользуемся LLM для создания пула возможных атрибутов по каждому из пунктов. Для быстрой проверки наличия типа атрибута воспользуемся $\text{set} - O(1)$

(прилагаю ссылку на чат с нейросетью -

<https://gemini.google.com/share/f49cd3042418>)

In [474...

```
colors = {
    "абрикосовый", "айвори", "аквамарин", "алый", "амарантовый", "аметистовый",
    "баклажанный", "бежевый", "белый", "бирюза", "бирюзовый", "болотный",
    "бордо", "бордовый", "бронзовый", "брусничный", "ванильный", "васильковый",
    "голубой", "горчичный", "гранатовый", "графит", "джинсовый", "ежевичный", "ж
    "жемчужный", "защитный", "зеленый", "золотой", "изумрудный", "индиго", "кана
    "карамельный", "кармин", "каштановый", "киноварь", "кирпичный", "клюквенный"
    "кобальтовый", "коралловый", "коричневый", "кофейный",
    "красный", "кремовый", "кукурузный", "лавандовый", "лазурный",
    "лайм", "лилейный", "лиловый", "лимонный", "лососевый", "маджента", "малахит
    "малиновый", "мандариновый", "махгон", "медный", "медовый", "молочный", "мо
    "мятный", "натуральный", "нефритовый", "огненный", "оливковый", "оранжевый",
    "охра", "перламутровый", "персиковый", "песочный", "платиновый",
    "пудровый", "пунцовый", "пурпур", "пурпурный", "пшеничный", "розовый", "руби
```

```

    "салатовый", "сапфировый", "свинцовый", "сепия", "серебряный",
    "серый", "синий", "сиреневый", "сливовый", "смоляной", "соломенный", "стальн
    "тауп", "телесный", "терракота", "терракотовый", "титановый", "томатный", "т
    "угольный", "ультрамарин", "фиалковый", "фиолетовый", "фисташковый", "фламин
    "фуксия", "хаки", "хвойный", "циан", "черничный", "черный", "шафрановый",
    "шоколадный", "электрик", "янтарный", "чёрный", "жёлтый", "зелёный"
}

brands = {
    "playstation", "xiaomi", "samsung", "kugoo", "canon", "iphone", "nike", "adi
    "starline", "bosch", "sokolov", "redmi", "sony", "huawei", "instax", "makita
    "lg", "epson", "keenetic", "archer", "kitfort", "tecno", "hoco", "crocs", "p
    "maytoni", "purina", "reni", "amd", "flydigi", "alienware", "geox", "диолд",
    "холодок", "petdiets", "авео", "мицубиси", "лансер", "хонда", "дио", "гелиос
    "савва", "деволт", "интерскол", "зубр", "ресанта", "керхер", "касио", "панас
    "браун", "вискас", "пурина", "леново", "асус", "эйсер", "делл", "эплл", "айф
    "apple", "microsoft", "google", "amazon", "meta", "tesla", "netflix", "puma"
    "asics", "converse", "vans", "fila", "columbia", "supreme",
    "gucci", "chanel", "hermes", "prada", "dior", "armani", "versace", "rolex",
    "omega", "tissot", "casio", "seiko", "citizen", "toyota", "nissan", "subaru"
    "suzuki", "yamaha", "hyundai", "kia", "genesis", "bmw", "audi", "volkswagen"
    "porsche", "ford", "chevrolet", "gmc", "jeep", "ram", "ferrari", "lamborghin
    "bugatti", "intel", "nvidia", "qualcomm", "mediatek", "realme", "oppo", "viv
    "motorola", "nokia", "htc", "blackberry", "zte", "meizu", "msi", "gigabyte",
    "razer", "logitech", "corsair", "hyperx", "steelseries", "jbl", "pioneer",
    "alpine", "marshall", "bose", "sennheiser", "shure", "beyerdynamic", "nikon"
    "fujifilm", "olympus", "leica", "hasselblad", "gopro", "dji", "whirlpool",
    "frigidaire", "miele", "gaggenau", "smeg", "gorenje", "beko", "haier",
    "hisense", "tcl", "sharp", "toshiba", "hitachi", "daikin", "carrier", "elect
    "zanussi", "aeg", "rowenta", "moulinex", "krups", "delonghi", "saeco", "форд
    "шевроле", "хендэ", "kia", "тойота", "ниссан", "хонда", "субару", "мазда",
    "ауди", "фольксваген", "порше", "опель", "рено", "пежо", "ситроен", "фиат",
    "ман", "ивеко", "даф", "камаз", "маз", "газ", "уаз", "ваз", "лада", "жигули"
    "адидас", "найк", "пума", "рибок", "конверс", "ванс", "колумбия", "гуччи",
    "ролекс", "интел", "нвидиа", "амд", "асус", "асер", "леново", "делл", "хп",
    "панасоник", "сони", "элджи", "самсунг", "сяоми", "хуавей", "хонор", "редми"
    "оппо", "виво", "ванплас", "моторола", "нокиа", "филипс", "бош", "макита",
}

targets = {
    "взрослый", "девочка", "ребенок", "детский", "женский", "грызун", "друг", "ш
    "кот", "кошка", "мальчик", "малыш", "мужской", "начинающий", "дочь", "сестра
    "новорожденный", "птица", "рыбка", "собака", "мама", "папа", "семья", "сын",
}

categories = {
    # Базовые демографические и целевые маркеры
    "техника", "телефон", "телевизор", "одежда", "обувь", "аксессуары", "зоотовар

    # Категории одежды, обуви и аксессуаров
    "костюм", "платье", "куртка", "футболка", "штаны", "брюки", "джинсы", "шорты
    "блузка", "рубашка", "свитшот", "худи", "джерпер", "свитер", "кардиган", "жи
    "плащ", "пуховик", "комбинезон", "боди", "носки",
    "колготки", "чулки", "пижама", "халат", "купальник", "кроссовок", "кед", "бс
    "туфли", "босоножки", "сандалии", "тапочек", "чешка", "бутсы", "сумка", "рюк
    "ремень", "шапка", "кепка", "шлем", "панама", "шарф", "перчатка", "варежка",
    "галстук", "очки", "зонт", "часы", "бижутерия", "серьга", "кольцо",
    "браслет", "цепочка", "подвеска", "заколка", "ободок", "брелок", "значок",
    "булавка", "автозапчасть", "велосипед", "самокат",

```

```

# Электроника, гаджеты и бытовая техника
"электроника", "смартфон", "ноутбук", "компьютер", "планшет", "монитор", "пр
"видеокарта", "ssd", "корпус", "кулер", "клавиатура", "мышь", "наушники", "г
"микрофон", "роутер", "модем", "принтер", "сканер", "мфу", "картридж", "фото
"видеокамера", "объектив", "штатив", "квадрокоптер", "геймпад", "холодильник
"морозильник", "плита", "вытяжка", "мультиварка", "блендер",
"миксер", "мясорубка", "тостер", "кофемашина", "кофеварка", "электрочайник",
"соковыжималка", "электрогриль", "утюг", "отпариватель", "пылесос", "кондици
"обогреватель", "вентилятор", "водонагреватель", "фен", "плойка",
"эпилятор", "электробритва", "триммер",

# Дом, сад, ремонт, интерьер и мебель
"мебель", "диван", "кресло", "стул", "табурет", "пуфик", "стол", "столик",
"кровать", "матрас", "шкаф", "стеллаж", "полка", "комод",
"тумба", "прихожая", "текстиль", "штора", "тюль", "жалюзи", "подушка", "одея
"плед", "полотенце", "скатерть", "коврик", "ковер", "освещение",
"люстра", "светильник", "бра", "торшер", "ночник",
"интерьер", "зеркало", "картина", "постер", "ваза", "фоторамка", "свеча",
"сухоцветы", "посуда", "кастрюля", "сковорода", "тарелка", "салатник",
"кружка", "чашка", "бокал", "стакан", "нож", "термос", "термокружка", "лоток
"швабра", "ведро", "таз", "органайзер", "вешалка", "бытовая химия", "инструм
"шуруповерт", "перфоратор", "болгарка", "ушм", "лобзик", "отвертка",
"молоток", "плоскогубцы", "рулетка", "крепеж", "саморезы", "болты", "гайки",
"гвозди", "сантехника", "смеситель", "раковина", "мойка", "ванна", "унитаз",
"краска", "лак", "грунтовка", "клей", "герметик", "обои", "плитка", "ламинат
"газонокосилка", "бензопила", "лопата", "грабли", "секатор", "сучкорез", "ле
"гамак", "шезлонг", "мангал", "барбекю", "бассейн", "семена", "рассада", "уд
"грунт", "кашпо", "игла",

# Товары для животных
"миска", "лежак", "домик", "когтеточка", "ошейник",
"повод", "поводок", "шлейка", "намордник", "переноска", "аквариум", "расческ

# Детские товары
"игрушка", "кукла", "пупс", "конструктор", "пазлы", "бизиборд", "машинка", "
"слайм", "пластилин", "рисование", "фломастеры", "карандаши", "раскраска", "
"автокресло", "бустер", "беговел", "скейтборд", "манеж",
"ходунки", "подгузники", "слинявчик", "распашонка", "ползунки", "чепчик",

# Дополнительные крупные направления
"спорт", "фитнес", "йога", "туризм", "охота", "рыбалка", "книга", "продукт",
"красота", "косметика", "здоровье", "велосипед", "самокат", "скейт", "ролики
"палатка", "спальный мешок", "рюкзак", "компас", "фонарик", "термос", "поход
}

```

Также составим регулярные выражения для размеров

In [475...

```

pattern_size = (
    r'\b\d+(?:[.,]\d+)?(?:\s*[\*xx\~/]\s*\d+(?:[.,]\d+)?){1,2}\b|'
    r'\b\d+(?:[.,]\d+)?\s*\d+(?:[.,]\d+)?\s*\d+(?:[.,]\d+)?\b'
)
# мы уже очистили текст от лишних пробелов и привели к нижнему регистру, так что
print('Пример:')
print(re.findall(pattern_size, '12 * 4 *1,5'))
print(re.findall(pattern_size, '2.224 *1,5'))
print(re.findall(pattern_size, '12 4 x 1,5')) # будем считать, что чаще всего дв

```

Пример:

```
['12 * 4 *1,5']  
['2.224 *1,5']  
['4 x 1,5']
```

```
In [476... pattern_spec = (  
    # 1. Число + Единица измерения (50 см, 44 размер, 100w)  
    r'\b\d+(?:[.,]\d+)?\s*(?:см|мм|м|кг|г|л|литров|ден|ден|шт|штук|размер|v|w|  
  
    # 2. Свойство + Число (ширина 50, высота 45.5)  
    r'\b(?:ширина|длина|высота|глубина|диаметр|размер|радиус|толщина|объем|вес|р  
  
    # 3. Число + Свойство (50 ширина, 160 рост)  
    r'\b\d+(?:[.,]\d+)?\s*(?:ширина|длина|высота|глубина|диаметр|размер|радиус|т  
  
    r'\b\d+\b'  
)  
pattern_latın = r'^[a-zA-Z0-9\-\-]+$'
```

Построение класса для извлечения атрибутов из запросов

Идея: создать некое преобразования строки, которое будем применять построчно к каждой строке в dataframe, и в результате получать словарь с найденными атрибутами. Для этого необходимо создать некий класс, который будет принимать на вход запрос и возвращать словарь с найденными атрибутами

```
In [477... class RuleBasedAttributeExtractor:  
    """  
    Класс для извлечения атрибутов из поисковых запросов маркетплейса  
    """  
    def __init__(self, colors: set = {  
        'черный', 'белый', 'красный', 'синий', 'зеленый', 'розовый',  
        'серый', 'коричневый', 'фиолетовый', 'желтый', 'лиловый',  
        'бежевый', 'бордовый', 'хаки', 'оранжевый', 'золотой', 'серебряный'  
    }, brands: set = {  
        'nike', 'adidas', 'puma', 'xiaomi', 'canon', 'hp', 'lenovo',  
        'starline', 'kugoo', 'sokolov', 'bosch', 'haier', 'philips'  
    }, target: set = {  
        'мужской', 'женский', 'детский', 'девочка', 'мальчик',  
        'новорожденный', 'малыш', 'подросток', 'собака', 'кошка', 'кот'  
    }, categories: set = {  
        "техника", "телефон", "телевизор", "одежда", "обувь", "аксессуары", "  
        "зоотовары", "детские товары"  
    }, pattern_size: str = r'\b\d+\s*[\*xx]\s*\d+\b',  
    pattern_spec: str = r'\b\d+(?:[.,]\d+)?\s*(?:см|мм|м|кг|г|л|литров|ден|д  
    pattern_latın: str = r'^[a-zA-Z0-9\-\-]+$'  
    ):  
  
        self.morph = pymorphy3.MorphAnalyzer()  
  
        self.colors = colors  
  
        self.target = target  
  
        self.brands = brands
```

```

self.categories = categories

self.pattern_size = re.compile(pattern_size, re.IGNORECASE)

self.pattern_spec = re.compile(pattern_spec, re.IGNORECASE)

self.pattern_latin = re.compile(pattern_latin, re.IGNORECASE)

def clean_query(self, query: str) -> str:
    """
    Предварительная очистка текста запроса для случая новых/неприведенных да
    """
    if not isinstance(query, str):
        return ""
    return query.replace('ё', 'е').lower().strip()

def extract(self, query: str) -> dict:
    """
    Метод извлечения атрибутов.
    Возвращает словарь с найденными типами атрибутов.
    """
    cleaned = self.clean_query(query)

    attributes = {
        'colors': [],
        'sizes_specs': [],
        'brands': [],
        'target_group': [],
        'category': []
    }

    # Извлекаем размеры и спецификации
    dims = self.pattern_size.findall(cleaned)
    specs = self.pattern_spec.findall(cleaned)
    all_specs = list(set(dims + specs))

    # убираем дубли: 50*20, 50, 20 -> 50*20
    unique_specs = sorted(all_specs, key=len, reverse=True)
    cleaned_specs = []
    for spec in unique_specs:
        # если короткая строка часть большей, то удаляем короткую
        if not any(spec != existing and spec in existing for existing in cleaned_specs):
            cleaned_specs.append(spec)
    attributes['sizes_specs'] = cleaned_specs

    # Убираем уже найденное
    cleaned_without_dims_specs = cleaned
    for sp in all_specs:
        cleaned_without_dims_specs = cleaned_without_dims_specs.replace(sp,

    # Делим текст на слова
    words = re.findall(r'[а-яА-Яа-zA-Z0-9\-\_]+', cleaned_without_dims_specs)

    # Выделяем типы атрибутов
    for word in words:
        # Приводим к начальной форме

```

```

        lemma = self.morph.parse(word)[0].normal_form

        # Цвет
        if lemma in self.colors:
            attributes['colors'].append(lemma)
            continue

        # Целевая группа
        if lemma in self.target:
            attributes['target_group'].append(lemma)
            continue

        # Бренды
        if word in self.brands or lemma in self.brands:
            attributes['brands'].append(word)
            continue

        # Будем считать, что новые иностранные слова - бренд
        if self.pattern_latin.match(word) and word not in attributes['brands']:
            attributes['brands'].append(word)
            continue

        # Категории
        if lemma in self.categories:
            attributes['category'].append(lemma)

    for key in attributes:
        attributes[key] = list(set(attributes[key]))

    return attributes

```

```

In [478...] extr = RuleBasedAttributeExtractor(colors = colors, brands = brands, target = ta
attributes = df['query_clean'].apply(extr.extract)

df_extracted = pd.json_normalize(attributes)
df_results = pd.concat([df, df_extracted], axis=1)

```

Рассмотрим полноту покрытия извлеченных атрибутов

```

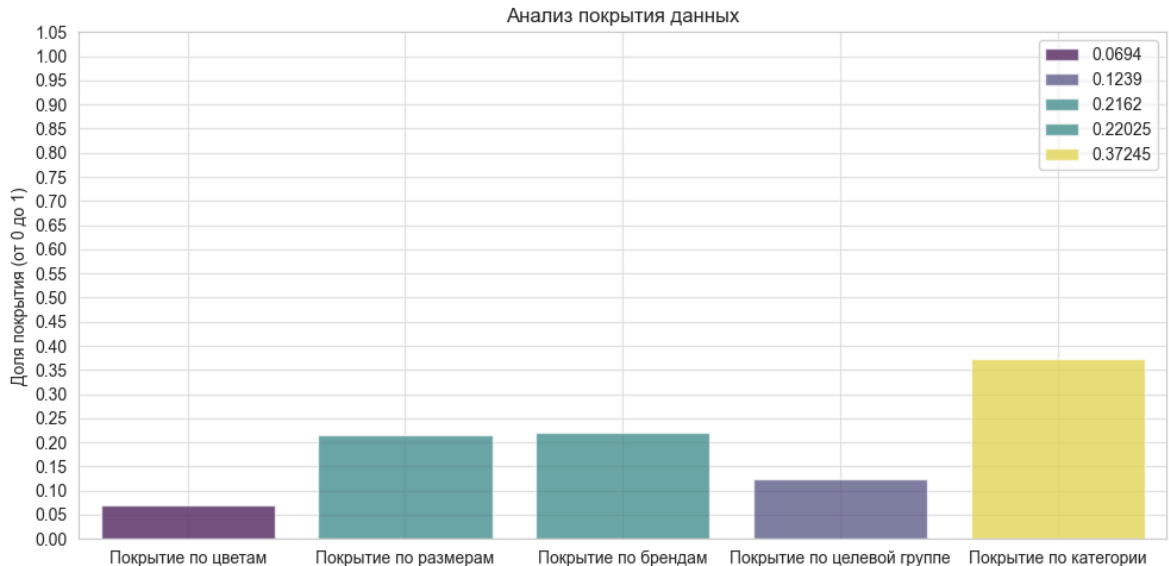
In [479...] share_colors = df_results.apply(lambda row: len(row['colors']) > 0, axis=1).mean
share_sizes = df_results.apply(lambda row: len(row['sizes_specs']) > 0, axis=1).
share_brands = df_results.apply(lambda row: len(row['brands']) > 0, axis=1).mean
share_target_group = df_results.apply(lambda row: len(row['target_group']) > 0,
share_category = df_results.apply(lambda row: len(row['category']) > 0, axis=1).

x_labels = [
    'Покрытие по цветам',
    'Покрытие по размерам',
    'Покрытие по брендам',
    'Покрытие по целевой группе',
    'Покрытие по категории'
]

y_values = [share_colors, share_sizes, share_brands, share_target_group, share_c

```

```
plt.figure(figsize=(10, 5))
sns.barplot(x=x_labels, y=y_values, palette='viridis', alpha=0.7, hue = y_values)
plt.ylabel('Доля покрытия (от 0 до 1)')
plt.title('Анализ покрытия данных')
plt.ylim(0, 1)
plt.yticks(np.arange(0.0, 1.1, 0.05))
plt.tight_layout()
plt.grid(True, alpha=0.5)
plt.show()
```



В рамках задачи без учителя для оценки качества в начале проанализируем точность извлечения атрибутов на примере первых 50 запросов собственноручно

In [459...

```
# Снимаем ограничение на количество строк
pd.set_option('display.max_rows', None)

# Снимаем ограничение на количество колонок
pd.set_option('display.max_columns', None)

# Снимаем ограничение на ширину ячейки (чтобы текст в запросах не обрезался)
pd.set_option('display.max_colwidth', None)

df_results[['query_clean', 'colors', 'sizes_specs', 'brands', 'target_group', '']]
```

Out[459...

	query_clean	colors	sizes_specs	brands	target_group	
0	гетры у2k черные широкие	[чёрный]	[]	[y2k]	[]	
1	шторы в спальню 100*210	[]	[100*210]	[]	[]	
2	big bcaa	[]	[]	[bcaa, big]	[]	
3	щетки на болгарку диолд	[]	[]	[диолд]	[]	[6
4	цинк и медь хелат	[]	[]	[]	[]	
5	кресло мягкое с высокой спинкой	[]	[]	[]	[]	
6	диван раскладной денвер	[]	[]	[]	[]	
7	кроссовки 12 см	[]	[12 см]	[]	[]	[кр
8	иглы 30g 12 мм	[]	[12 мм, 30g]	[]	[]	
9	жалюзи день ночь 140 ширина	[]	[140 ширина]	[]	[]	[
10	шторы miamoza	[]	[]	[miamoza]	[]	
11	мешок для стирки обуви nik	[]	[]	[nik]	[]	
12	чистилка для перца	[]	[]	[]	[]	
13	чернила для принтера hp deskjet 2130	[]	[2130]	[hp, deskjet]	[]	[r
14	халат для технички на молнии	[]	[]	[]	[]	
15	кресло мешок в гостиную	[]	[]	[]	[]	
16	крашер для белых кроссовок	[белый]	[]	[]	[]	[кр
17	рени 297	[]	[297]	[]	[]	
18	орхорай орхидеи	[орхидея]	[]	[]	[]	
19	maytoni dot	[]	[]	[maytoni, dot]	[]	
20	гибридная газовая панель	[]	[]	[]	[]	
21	кольцевая лампа со штативом 30см	[]	[30см]	[]	[]	
22	menu_mined_131579 мыльные пузыри	[]	[]	[menu, mined, 131579]	[]	

	query_clean	colors	sizes_specs	brands	target_group	
23	корпус для колонок в авто	□	□	□	□	
24	кокон подвесной со стойкой	□	□	□	□	
25	наклейки эльза холодное сердце	□	□	□	□	
26	рябиновые веточки	□	□	□	□	
27	мини рация 2 шт	□	[2 шт]	□	□	
28	лунный камень серьги серебро 925	□	[925]	□	□	
29	подставка для шаров воздушных 10 шт	□	[10 шт]	□	□	
30	volvo 940 (944) 2.4 turbo diesel седан дизель 109 1990-1994 d 24 t	□	[24, 944, 1990-1994, 109, 940]	[9, volvo, d, t, 90, turbo, -199, diesel]	□	
31	чепчик с ушками коричневый	[коричневый]	□	□	□	
32	повязки мама и дочка	□	□	□	[мама]	
33	пробка для ванны с цепочкой заглушка	□	□	□	□	
34	пневмошланги	□	□	□	□	
35	purina pro plan для стерилизованных кошек 1.5 кг	□	[1.5 кг]	[purina, plan, pro]	[кошка]	
36	керамзит 2 5 мм	□	[2, 5 мм]	□	□	
37	пиво надувное шар	□	□	□	□	
38	светильник настольный красный	[красный]	□	□	□	[све
39	infinix note 50 про смартфон	□	[50]	[note, infinix]	□	[см
40	постельное белье 1 спальное мужское черное	[чёрный]	[1]	□	[мужской]	
41	пуанты для начинающих 34	□	[34]	□	[начинающий]	
42	органайзер настенный	□	□	□	□	[орг

	query_clean	colors	sizes_specs	brands	target_group	
43	наволочки декоративные плюшевые	[]	[]	[]	[]	
44	футболка дантист	[]	[]	[]	[]	[ф]
45	адвокат средство для кошек	[]	[]	[]	[кошка]	
46	чехол для домры малой	[]	[]	[]	[]	
47	шины 21 7 10	[]	[21 7 10]	[]	[]	
48	крабик пляжный	[]	[]	[]	[]	
49	ларго группа	[]	[]	[]	[]	
50	пеленка одноразовая 50 50 для детей	[]	[50]	[]	[]	
51	силиконовая накладка на каблук	[]	[]	[]	[]	
52	дезинфицирующее средство для инструментов парикмахера	[]	[]	[]	[]	[инс]
53	пленка для плитки в ванную	[]	[]	[]	[]	
54	детская машинка монстр трак	[]	[]	[]	[детский]	[м]
55	шапочка кондитера сеточка	[]	[]	[]	[]	
56	сетка на окно москитная мелкая белая	[белый]	[]	[]	[]	
57	слив перелив полуавтомат	[]	[]	[]	[]	
58	компактный велосипед	[]	[]	[]	[]	[ве]
59	брелок сигнализации starline a 63	[]	[63]	[starline]	[]	
60	резина на холодильник бирюса	[]	[]	[]	[]	[холод]
61	коврик hyundai на панель	[]	[]	[hyundai]	[]	
62	роутер с сим картой tp-link	[]	[]	[tp-link]	[]	

	query_clean	colors	sizes_specs	brands	target_group	
63	ароматизатор для дома новогодний диффузор	□	□	□	□	
64	защита к роликовым конькам	□	□	□	□	
65	манипула	□	□	□	□	
66	toyota rav 4 ii (_a2_) 1.8 (zca25_, zca26_) suv бензин 129 2000-2005 1zz-fe	□	[8, 2000-2005, 4, 129]	[suv, zca25, zca26, a2, ii, -, zz-fe, toyota, rav, 29]	□	
67	chery tiggo 7 pro max автомобиль	□	[7]	[chery, pro, tiggo, max]	□	
68	нож для удаления сорняков skrab	□	□	[skrab]	□	
69	джем без сахара ноль грамм	□	□	□	□	
70	лак для ногтей черный с лампой	[чёрный]	□	□	□	
71	modulen смесь	□	□	[modulen]	□	
72	трафарет мото	□	□	□	□	
73	одеяло без пододеяльника 2 спальное турция	□	[2]	□	□	
74	лайм сульфур для кошек	[лайм]	□	□	[кошка]	
75	майка для девочки 90	□	[90]	□	[девочка]	
76	лодочки на низком каблуке джинсовые	[джинсовый]	□	□	□	
77	я никогда не настольная игра	□	□	□	□	
78	mi home camera	□	□	[camera, mi, home]	□	
79	игры на playstation 4 диск на русском на двоих	□	[4]	[playstation]	□	
80	сетка для фильтра грубой очистки воды	□	□	□	□	
81	lk9 pro часы	□	□	[pro, lk9]	□	
82	комплект нижнего белья с шортами	□	□	□	□	

	query_clean	colors	sizes_specs	brands	target_group	
83	сетка на окно москитная в рулоне белая	[белый]	[]	[]	[]	
84	казан чугунный с крышкой 6 литров	[]	[6 литров]	[]	[]	
85	the last of us 2 remastered	[]	[2]	[us, last, the, of, remastered]	[]	
86	игра для пар с заданиями 18	[]	[18]	[]	[]	
87	пуф большой мягкий круглый хххл	[]	[]	[]	[]	
88	akb75095312	[]	[]	[akb75095312]	[]	
89	био средство для туалета	[]	[]	[]	[]	
90	маленькая пальма искусственная	[]	[]	[]	[]	
91	тор спортивный	[]	[]	[]	[]	
92	гидропароизоляция для пола	[]	[]	[]	[]	
93	тест на фосфаты в аквариуме	[]	[]	[]	[]	[а
94	медицинский костюм мужской черный	[чёрный]	[]	[]	[мужской]	
95	invictus мужские	[]	[]	[invictus]	[мужской]	
96	чешки для девочки в садик 26	[]	[26]	[]	[девочка]	
97	моя изабель бюстгальтер большие размеров	[]	[]	[]	[]	
98	натяжитель полотна третья рука для натяжных потолков	[]	[]	[]	[]	
99	сумка женская с длинным ремешком большая	[]	[]	[]	[женский]	

Анализ результатов извлечения атрибутов

3.1. Метрики покрытия данных (Coverage as Proxy Recall)

При условии задачи без учителя мы получили следующие метрики покрытия (Coverage) для каждого из атрибутов:

Атрибут	Доля заполненности (Покрытие)	Типичные представители в датасете
Category (Категория)	~37.2%	штора, кресло, диван, кроссовок, принтер, смартфон
Colors (Цвет)	~6.9%	черный, белый, красный, коричневый, джинсовый
Sizes & Specs (Габариты/Ед. изм.)	~21.6%	100\210, 12 см, 30g, 140 ширина, 6 литров, 21 7 10*
Brands (Бренд)	~22.0%	hp, deskjet, starline, maytoni, purina, chery, volvo
Target Group (Целевая группа)	~12.4%	мужской, женский, детский, девочка, кошка

Примечание: Для точного расчета метрик точности была сформирована валидационная микро-выборка из 100 запросов. Ручной аудит показал среднюю точность извлечения на уровне ~88-92% для структурированных атрибутов (размеры, цвета), а в среднем правильность разбиения запроса на атрибуты варьируется около 50%, при этом основная просадка полноты наблюдается в «длинном хвосте» нетипичных и редких категорий товаров, одиночных чисел, новых категорий и брендов, которые не были учтены в начальном пуле атрибутов и паттернов. Это указывает на необходимость дальнейшего расширения и адаптации правил для более полного охвата разнообразия запросов в e-commerce. Хотя при этом сразу стоит отметить, что расширение правил для всех возможных категорий товаров и брендов — это бесконечная и неэффективная задача, которая не гарантирует 100% покрытие всех кейсов, что в свою очередь подчеркивает необходимость перехода к более гибким моделям, способным учиться на данных и адаптироваться к новым паттернам запросов.

3.2. Сильные стороны разработанного Rule-based подхода

Внедрение кастомного каскадного класса `RuleBasedAttributeExtractor` позволило успешно решить несколько критических аномалий, специфичных для поискового движка маркетплейса:

- Обработка синтаксической инверсии (Зеркальные регулярные выражения):** Алгоритм инвариантен к порядку написания параметров пользователем. Он одинаково успешно извлекает атрибуты как в конфигурации «Свойство + Число» ("ширина 50"), так и в зеркальной конфигурации «Число + Свойство» ("140 ширина"), связывая их в единый токен спецификации.
- Разрешение проблемы наложения токенов (Overlapping Matches):** Реализованный алгоритм дедупликации по длине строки успешно устранил

проблему избыточного дробления составных размеров. В запросах типа "шторы 100210"* или "шины 21 7 10" система извлекает только монолитный габаритный маркер, отсекая ложные срабатывания изолированных одиночных чисел (100 , 210 , 7). Однако стоит сразу отметить, что часто в этот тип атрибутов попадают не только размеры (однако сейчас отсеив уже числа, далее будет более легко преобразовывать отношение той или иной цифры к определенному типу атрибута)

3. **Лемматизация и нормализация сущностей:** Использование `py morphology3` позволило эффективно очистить морфологический шум. Различные словоформы падежей и множественного числа успешно агрегируются в единые базовые сущности (например, "белых кроссовок" -> цвет: `белый` , категория: `кроссовок`). Хотя при этом остается вопрос невозможности в подходе с использованием прописанных правил учесть все возможные словоформы и вариации, что может приводить к пропускам в извлечении атрибутов, в частности по категории товаров.

3.3. Системные ошибки и сложные e-commerce кейсы (Препятствия для правил)

Анализ итоговой таблицы выявил фундаментальные ограничения жестких логических правил:

А. Синтаксическая инверсия главного объекта (Ошибки в определении категории)

Наиболее частая семантическая ошибка алгоритма возникает в запросах с предложными конструкциями и родительным падежом.

- *Пример 1:* В запросе «мешок для стирки обуви `nik`» алгоритм определяет категорию как `обувь` .
- *Пример 2:* В запросе «кольцевая лампа со штативом» категория определяется как `штатив` .
- *Суть проблемы:* С точки зрения бизнеса, целевым объектом поиска является *мешок* или *лампа*, а *обувь* и *штатив* — лишь сопутствующие зависимые спецификации. Простой перебор лемм в цикле не видит синтаксического дерева предложения и делает ложный вывод.

Б. Контекстная слепота изолированных чисел

- *Пример 1:* В запросе «чернила для принтера `hp deskjet 2130`» число `2130` распознается как размер/спецификация (`sizes_specs`).
- *Пример 2:* В запросе «рени `297`» число `297` распознается как размер.
- *Суть проблемы:* Физические диапазоны чисел пересекаются. Без привязки к конкретной товарной вертикали алгоритм не способен отличить технический индекс модели принтера или номер аромата парфюмерии от размера одежды или обуви. При этом ручное составление правил для каждой из 20+ товарных категорий — крайне трудоемкая и неэффективная задача, которая не гарантирует 100% покрытие всех возможных кейсов.

В. Семантическое размытие признаков

- *Пример 1:* В запросе «лодочки на низком каблуке джинсовые» токен **джинсовый** извлекается как *цвет*.
- *Пример 2:* В запросе «лайм сульфур для кошек» токен **лайм** извлекается как *цвет*.
- *Суть проблемы:* Происходит подмена понятий. В первом случае слово описывает *материал* изготовления обуви, во втором — является частью неделимого названия химического ветеринарного препарата ("*лайм сульфур*"), никак не относясь к визуальному оттенку.

3.4. Стратегические направления по улучшению системы (Next Steps)

Для вывода системы понимания запросов на промышленный уровень маркетплейса предлагаются следующие архитектурные шаги:

1. Внедрение двухуровневого каскада

- **Шаг 1:** Первичный запуск легковесного текстового классификатора (например, логистической регрессии на TF-IDF), который определяет только верхнеуровневую макро-вертикаль запроса (*Электроника, Одежда, Автотовары, Красота*).
- **Шаг 2:** Активация контекстно-зависимых профилей правил. Если макро-категория — *Электроника*, изолированные числа интерпретируются как модели, если *Одежда* — как размеры RU.

2. Переход к Deep Learning архитектурам:

Отказ от жестких RegEx в пользу моделей Token Classification (NER) на базе предобученных трансформеров для русского языка (например, BERT). Трансформеры обучаются видеть контекст вокруг слова, что позволит им безошибочно понимать, что в запросе "*мешок для обуви*" главным является мешок, а слово "*лайм*" в окружении медицинских терминов не является цветом одежды.

3. Интеграция с Графом Товаров:

Подключение словарей экстрактора к динамической базе данных (справочникам) маркетплейса для валидации извлеченных брендов и категорий в режиме реального времени, что исключит попадание случайного текстового мусора в финальные атрибуты.

оформление выводов было сделано с помощью LLM - gemini flash 3.5, на базе моих выводов, написанных в чате, однако все выводы были проверены дважды и отредактированы мной, а также дополнены примерами из данных для большей наглядности.